

When Do Corrective Features Help? An Agent for Corrective Feature Discovery on Black-Box Forecasters

Fangxin Wang¹, Ziyi Zhang², Diyi Zhuang³, Langzhou He¹, Shiyu Wang,
Baichuan Mo⁴, Philip S. Yu¹

¹University of Illinois Chicago, ²Texas A&M University,

³Massachusetts Institute of Technology, ⁴Tsinghua University

Abstract

Frozen pretrained forecasters often fail in structured, recurring ways that are costly to repair through fine-tuning. We study *corrective feature discovery*: mining interpretable features of the frozen forecaster’s residual to drive a lightweight post-hoc corrector. Prior automated feature engineering models the data-generating process; corrective features instead model the *model-failure* process. We present CRAFTER (Corrective Residual Agent with Feature-based Temporal Exploration and Reasoning), which keeps the backbone frozen and mines its residual with two generators of different character: a compositional search over the raw input channels, and a large language model (LLM) that proposes named feature combinations, binary flags, and short executable code. A single validation-grounded gate accepts or rejects every candidate blind to its origin, and a validation-selected corrector applies the survivors or leaves the forecast unchanged. The same source-blindness lets prior feature-engineering systems run through the identical pipeline, so CRAFTER doubles as an instrument that attributes forecast changes to the feature source alone. Across six public datasets and six frozen backbones, CRAFTER surpasses every dedicated feature-engineering system at every feature budget, roughly doubling the corrector-only lift and cutting the weakest backbones’ error by up to 27%. The gains are robust to the LLM backend and persist on top of fine-tuned backbones.

1 Introduction

Large-scale forecasting is now served by pretrained backbones deployed *frozen* (Liu et al., 2024; Ansari et al., 2024; Liu et al., 2025a). An e-commerce platform or grocery chain predicts sales for hundreds of thousands of store–product series from a single model, because fine-tuning one per item or regime is infeasible. The frozen model still fails in structured, recurring ways: it misses promotions,

lags regime shifts, and underprices event spikes. These errors are business-critical and must be interpretable to act on, so the question is not how to train a better forecaster but how to *correct* a deployed one cheaply, without modifying it.

A natural way to correct a fixed forecaster is to add external, interpretable features around it. This connects the problem to automated feature engineering, but with a different target: once the backbone is frozen, the object of discovery changes from the target process itself to the residual of a particular deployed model. The relevant question is not which regularities help predict the future, but which remain unmodeled by this backbone. We therefore study *corrective feature discovery*: automatically finding interpretable features of the residual, the signal of where and how a particular frozen forecaster fails. The residual is not leftover noise a priori—it carries structure induced by the backbone’s blind spots, concentrated around promotions, rare events, and local regimes—so a corrective feature is model-dependent and doubles as a diagnosis: each accepted feature names a mechanism the deployed model misses.

Two properties make the discovery hard. First, the most valuable corrective features are *named* from domain semantics rather than enumerated from the raw schema; where they matter, a statistical feature bank and a syntactic search improve the forecast only modestly, while named features close most of the remaining gap (§5.1). Second, a feature’s value is *regime-dependent*: it concentrates where the backbone errs and vanishes where the backbone is already accurate, so the same source helps on one backbone–dataset cell and is inert or harmful on another. Whether a feature source or corrector configuration helps therefore depends on the regime, and this dependence has not been characterized systematically.

CRAFTER addresses the method and the characterization together. It keeps the backbone frozen

and mines the residual with two generators of different character: a compositional search over the raw channels, and an LLM that proposes named combinations, binary flags, and short executable code. A single validation-grounded gate admits candidates blind to their origin, and a validation-selected corrector applies the survivors or leaves the forecast unchanged when none help. This source-blind design merges two heterogeneous generators into one method, and also turns the pipeline into an instrument for fairly evaluating external feature-engineering system.

We evaluate CRAFT on six public datasets and six frozen backbone families using a rolling-origin, multi-seed protocol. All methods are run in the same evaluation harness and, where applicable, with the same LLM backend. Across this controlled comparison, CRAFT consistently outperforms three dedicated feature-engineering systems. This advantage is robust to feature budget, remains under a second LLM backend, and also holds when the corrected backbone is fine-tuned rather than used zero-shot. On the two datasets where we test temporal reuse, the discovered features transfer across backtest windows. Finally, we identify the limits of residual correction: in saturated cells, no feature source yields a reliable improvement.

Contributions. (C1) **A source-blind framework for corrective feature discovery.** We formulate feature discovery for a frozen forecaster as residual correction, and instantiate it with a single validation-grounded gate that admits heterogeneous sources blind to their origin. The framework is both a deployable correction layer and an evaluation instrument: any external feature-engineering system runs through the identical gate, corrector, and budget, so a difference in the forecast is attributable to the feature source alone. (C2) **A characterization of when corrective features help.** Across six datasets and six backbone families we map where residual correction is beneficial, inert, or harmful. Correction pays in proportion to the residual’s exploitable structure: large on weak backbones with informative forecast covariates, where LLM-named features are the differentiator, and neutral once a strong backbone saturates the series. Budget amplifies that margin rather than creating it.

2 Related Work

Frozen foundation forecasters. Pretrained time-series models are often deployed zero-shot, with a single backbone served frozen across many series (Ansari et al., 2024; Liu et al., 2024, 2025a). This avoids per-series fine-tuning, but the frozen model leaves a backbone-specific residual it cannot remove on its own. We take that residual as the object to model and leave the backbone untouched, consistent with broader lightweight post-training approaches that add capabilities to frozen foundation models without updating their pretrained weights (Houlsby et al., 2019; Hu et al., 2021; He et al., 2025).

Automated and LLM-based feature engineering. A long line of work builds features to improve a trainable predictor. Statistical banks extract many candidate features and select among them (Christ et al., 2018; Cerqueira et al., 2020; Costa, 2021), and search-based engines compose operators over the raw schema with reinforcement learning (Khurana et al., 2018), Monte-Carlo tree search (Huang et al., 2022), or expert-level generation (Zhang et al., 2022). A recent line prompts an LLM to write feature code from dataset context (Hollmann et al., 2023; Nam et al., 2024; Abhyankar et al., 2025; Murray et al., 2025), and AutoCT pairs an LLM with tree search for tabular features (Liu et al., 2025b). Two things separate this work from ours. Its objective is *predictive*: the features train a downstream model rather than explain a fixed one. And its strongest LLM methods target *tabular* data. We run three of these systems as feature generators inside our own pipeline (§4), which puts every source on one footing.

Correcting a frozen forecaster. Modeling the error of a fixed forecaster with a second learner is a classic idea (Zhang, 2003). Recent methods correct a deployed forecaster without retraining it. Post-Training Corrections picks a sequence of corrections from a fixed pool (Cherkaoui et al., 2025). The δ -Adapter learns a sparse input mask and a bounded residual head, and uses no LLM (Liang et al., 2026). Other methods adapt a frozen forecaster online or through a memory module (Lyu et al., 2026; Dai et al., 2026a), and AutoGluon-TimeSeries regresses on raw covariates inside its pipeline (Shchur et al., 2023). These methods correct the forecast, but they act on inputs that are already given. None *discovers* new interpretable

features of the residual.

Discovering residual features to correct. Closest to our setting is NSR-Boost, which keeps a model frozen and uses an LLM to write symbolic residual experts for an aggregator to combine (Dai et al., 2026b). It is tabular, it uses the LLM as its only generator, and it runs no search. Work on forecasting explanation and interpretable correction analyzes or adjusts model behavior, but does not discover corrective features (Wang et al., 2023; Lopez and Sobieczky, 2024). CRAFTER differs in three ways. It mines the residual of a frozen *forecaster*. It draws candidates from two different generators, a compositional search and an LLM, and admits them through one validation gate that is blind to their source. Because the gate is source-blind, the same pipeline also measures *which* source and configuration help in *which* regime, which the work above does not study.

3 Method

We present CRAFTER (Corrective Residual Agent with Feature-based Temporal Exploration and Reasoning), an agent for *corrective feature discovery*. It leaves the forecasting backbone frozen and mines interpretable features of its residual, the signal of *where* and *how* the backbone fails, to drive a lightweight post-hoc corrector. This section describes the framework and the single source-blind gate at its core (§3.1), its two feature generators, *Exploration* (a compositional search) and *Reasoning* (a language model), with their optional coupling (§3.2), the gate (§3.3), and the validation-selected corrector (§3.4).

3.1 Overview

Illustrated in Fig. 1, CRAFTER corrects a frozen forecaster without modifying it. Given the backbone forecast $\hat{y}$ over a horizon, it mines interpretable features of the residual and fits a lightweight gradient-boosted-tree corrector on them, which then adjusts the forecast or leaves it unchanged. Quality is measured by the weighted mean absolute percentage error (wMAPE) of the horizon total (the corrector target and hyperparameters are in App. A and D). Covariates enter only as corrector features, never as backbone inputs; §5.4 shows this routing is what makes the gain possible.

Every candidate feature is a *structured specification*: a typed record drawn from a fixed grammar (App. C) that applies transforms and an oper-

ator to a few input channels and compiles to one numeric column. Specifications are searchable, de-duplicable across rounds, and interpretable by name, so a run can be audited feature by feature. CRAFTER mines the residual with two generators of different character—an atomic search over the raw channels and an LLM that proposes named features (§3.2)—and routes both into a single *source-blind* acceptance gate (§3.3) that keeps a candidate only when it explains validation error the corrector cannot already account for, whichever generator produced it.

CRAFTER runs for R rounds (Alg. S1, App. B). Each round fits the corrector, diagnoses where it still errs on a held-out validation block, mines new candidates from both generators, and keeps those that pass the gate. A validation-selected corrector then applies the survivors or leaves the backbone unchanged when none help (§3.4).

3.2 Feature generators

CRAFTER mines the residual with two generators of different character. A compositional search composes the raw channels into new features, and an LLM names mechanisms the raw schema does not contain. Both feed candidates into the single gate of §3.3, which decides what survives.

Compositional search. The first generator builds *compositional* features in the tradition of operator-based automated feature engineering (Cerqueira et al., 2020; Costa, 2021; Zhang et al., 2022), named **atomic** in this paper. A feature applies a transform to one channel, or an operator to two transformed channels, drawn from a fixed vocabulary of $T=15$ transforms and $B=8$ operators (App. C). The depth-one and depth-two expressions number far too many to score exhaustively, about 6×10^5 for $C \approx 25$ channels, so CRAFTER searches. It uses a Monte-Carlo tree search (Kocsis and Szepesvári, 2006) whose nodes are UCB1 bandits (Auer et al., 2002). Channels are grouped into mechanism *families* such as volatility or trend; a family pools its channels, so one reward updates the whole mechanism and the search finds productive families from few samples before refining to a single channel. A root-to-leaf path picks a family and arity, then the channels, then the operator, each node conditioned on the choices above it (App. C). An arm is credited by two signals: a cheap proxy at scoring time, the rank correlation ρ between the candidate and the still-unexplained val-

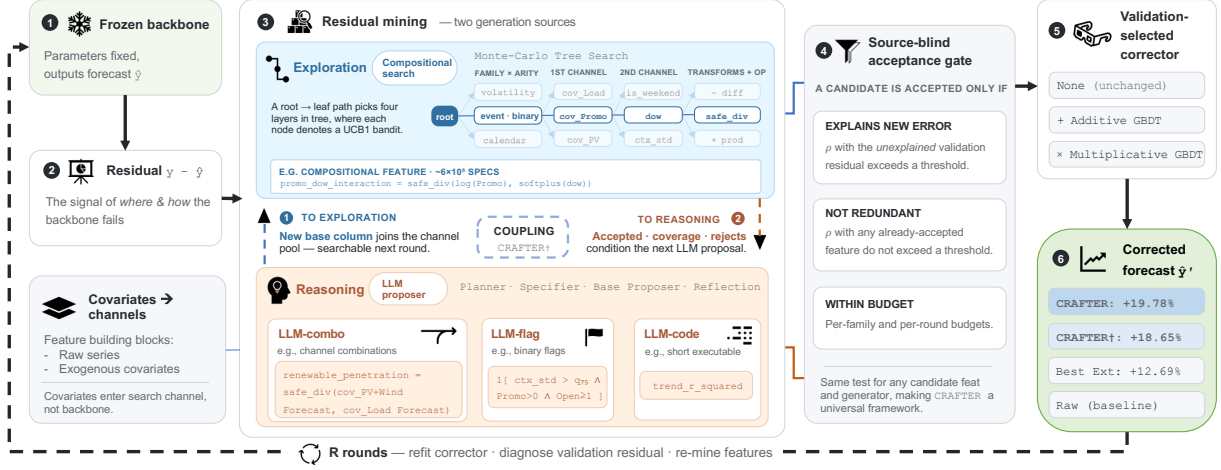


Figure 1: **The CRAFT framework.** The forecaster stays frozen. Two generators mine its residual for structured feature specifications: a compositional Monte-Carlo tree search and an LLM proposer, optionally coupled (CRAFT[†]). A source-blind gate admits a candidate only when it explains validation error the corrector cannot. A validation-selected corrector based on correlation ρ and budget then applies the survivors, or leaves the forecast unchanged. The loop runs R rounds.

validation residual, and the realized gain in validation wMAPE after the corrector is refit. The bandit selects arms by UCB1 and updates each arm’s value by the incremental sample mean of these rewards; the selection, update, and reward rules are Eq. (S3)–(S5) in App. D. The realized gain ties the search to the deployed model: a feature is worth what keeping it actually buys.

LLM-proposed features. The search only recombines existing channels; the LLM *invents* features that name mechanisms the schema lacks. Organized into roles and queried a few times per round, it reasons over the run’s own evidence, not generic knowledge alone: which features the gate accepted, which it rejected, and which mechanism families are still uncovered. A PLANNER reads this state and proposes a few exploration *directions*. A SPECIFIER turns each direction into candidate specs built from existing channels, surfacing semantic compositions a syntactic search would not prioritize: on the EPF electricity-price series it composes a `renewable_penetration`, renewable generation over total load, which captures the price suppression a covariate-blind backbone misses. A BASE PROPOSER goes further and invents new named base columns for mechanisms the schema lacks, which join the channel pool and become searchable next round. A REFLECTION call revises the next round’s directions when few candidates survive the gate. The LLM proposes three kinds of features: 1) **LLM-combo**: combinations of ex-

isting channels, 2) **LLM-flag**: binary flags, and 3) **LLM-code**: short executable code (accepted examples of every kind in App. K). LLM only *proposes*: which candidates survive is decided by the gate of §3.3, not by the model. All role prompts are in App. H.

Coupling the two generators. The two can also run as a coupled loop. A new base column invented by the LLM, once it passes the gate, joins the channel pool, so the search can compose it with other channels in later rounds. Conversely, each round’s merged accepted set, family coverage, and rejected candidates condition the next LLM proposal, steering it toward thin families and away from past rejects. Decided by whether allowing coupling, we have both CRAFT and CRAFT[†] (coupled) evaluated in the experiments.

3.3 The source-blind acceptance gate

Both generators feed one gate, and that gate is what makes CRAFT a single method rather than two pipelines. A candidate is accepted only if it explains validation error the corrector cannot *already* account for: the Spearman correlation between the candidate and the unexplained validation residual must exceed a threshold τ , the candidate must not be collinear with an already-accepted feature, and per-family and per-round budgets must hold. The same test applies to every candidate, compositional or semantic. Deciding acceptance on one validation-grounded scale makes the two

generators commensurable without either reading the other’s internal scores: the merged pool self-selects against the residual, not against any generator’s private confidence.

The gate is agnostic to a feature’s origin, with a consequence beyond merging the two generators. Any external feature-engineering system runs through the identical pipeline by swapping only the generator, with the gate, corrector, and selection unchanged. CRAFT then doubles as an *instrument*: a difference in the final forecast is attributable to the feature source, because nothing else moves (§5.1). The same source-blindness that unifies the two generators keeps that comparison fair.

3.4 Validation-selected corrector

The features that pass the gate feed a corrector. CRAFT fits a small *corrector set*—an identity (NONE) map plus an additive and a multiplicative gradient-boosted corrector—and keeps the one with the best validation wMAPE. Trees are chosen because the residual is nonlinear and regime-dependent and because per-feature importances keep the correction auditable (App. D). The NONE option floors the risk *on validation* (a round never ships a corrector worse there than leaving the backbone alone), making correction an asymmetric bet. This is a validation-time safeguard, not a test guarantee: under rolling-origin shift a low-headroom cell can still regress out-of-sample (e.g., BIZITOB/Chronos, §5.1).

4 Experimental Setup

Datasets. We use six public forecasting datasets spanning the regimes where covariate information varies in kind and strength (listed in the order used throughout): EPF-DE (day-ahead electricity prices; a single long series with *forecast-type* renewable covariates) (Lago et al., 2021), two promotion-driven retail panels (ROSSMANN, ROHLIK) (Kaggle, 2015, 2024), an hourly business-operations panel (BIZITOB-L2C) (Palaskar et al., 2024), a store-sales panel (FAVORITA) (Kaggle, 2017), and the M5 weekly competition data (M5) (Makridakis et al., 2022). Together they cover single- vs. many-series, covariate-rich vs. covariate-poor, and weak- vs. saturated-backbone settings.

Frozen backbones. Our grid corrects six pre-trained backbones used zero-shot, listed in order of release: Timer (Liu et al., 2024), Chronos (Ansari et al., 2024), Moirai (Woo et al., 2024), and

Toto (Cohen et al., 2024), followed by Chronos-2 (Ansari et al., 2025) and Moirai-2.0 (Liu et al., 2025a), spanning the first foundation-model wave to current leaders. Chronos and Timer are primarily univariate or channel-independent forecasters; Moirai and Toto can handle multivariate time-series inputs; and Chronos-2 explicitly exposes a covariate-aware forecasting interface. In every grid we nonetheless run *all* backbones covariate-blind, so the backbone forecast never sees the covariates and every method corrects the *same* residual—any covariate value is then attributable to the corrector, not the backbone (routing them into the backbone instead is at best neutral; §5.4). The grid is $6 \times 6 = 36$ cells.

Protocol. We use a rolling-origin backtest: on cached forecasts we re-cut three expanding train:val:test splits (6:1:1, 7:1:1, 8:1:1) with no test leakage, validate on three blocked CV windows, run $R=3$ rounds, and report last-round test wMAPE averaged over the three splits and three seeds (nine runs per cell). We assess significance with a one-sided Wilcoxon signed-rank test over the paired cells and decompose run-to-run variance into seed and split components (App. D); split variance dominates. The public datasets may appear in either LLM’s pretraining corpus, but the leakage surface is small: the LLM emits only feature *specifications* and code reading past history and the backbone forecast, never test-period values. Because seeds do not change the cached forecasts, the grid is GPU-free and cheap; compute and timing details are in App. D.

Baselines and backend. Every feature-engineering method—both CRAFT variants and the external systems—uses the same GPT-5.2 backend and an identical harness (same cached residual, same corrector set, same validation top- K selection), so only the feature *generator* differs and the comparison is fair by construction. We report two CRAFT variants, the two rightmost (highlighted) columns of App. Table S12: CRAFT, the full system, and CRAFT[†], which additionally enables the two source-coupling channels of §3.2. The baselines are: the frozen backbone (RAW); a backbone fine-tune (FT); a uniform head-only fine-tune of every backbone, with a stronger full-parameter Moirai variant in App. F); a covariate-only corrector (COV) and the no-LLM corrector (CRAFT-noLLM), which ablate the LLM and the search; and three

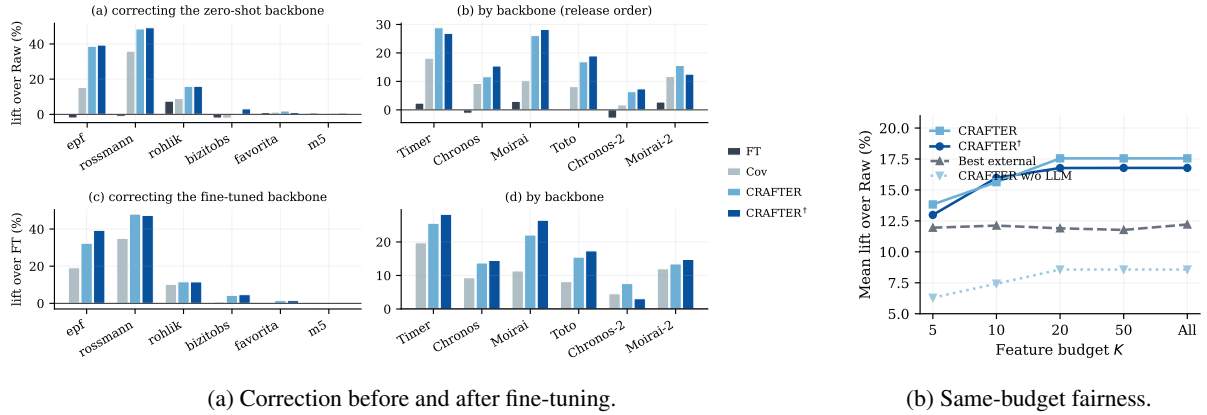


Figure 2: **Correction holds across training and feature budgets.** Mean lift in test wMAPE (% , higher is better) over the 6 datasets. Colours are shared across panels; the two CRAFT variants share a hue. **(a)** Correcting the zero-shot backbone (lift over RAW) and the fine-tuned one (lift over FT). CRAFT helps in both, with the largest gains on weak backbones. **(b)** Lift over RAW against budget K ; “All” is uncapped. Both CRAFT variants beat the best external at every K ; the no-LLM corrector stays below. Per-cell results in App. Table S12.

external feature-engineering systems adapted to our setting—TSFRESH (a statistical feature bank) (Christ et al., 2018), CAAFE (LLM-written feature code) (Hollmann et al., 2023), and LLM-FE (LLM-guided evolutionary search) (Abhyankar et al., 2025). App. E states what each adaptation keeps and changes.

5 Results

Corrective feature discovery improves a frozen forecaster without retraining it, and the size of that improvement is governed by residual headroom rather than by backbone identity, feature budget, or LLM backend. We evaluate on six datasets and six pretrained backbones under one rolling-origin, multi-seed protocol in which every feature-engineering method passes the same validation gate and downstream corrector, so measured differences trace to the discovered features rather than to the corrector (App. D; full per-cell grid in Table S12). This section establishes five points: CRAFT is state of the art against prior feature engineering and helps weak backbones most (§5.1); the wrapper generalizes across backbone families (§5.3) and survives non-standard input paths (§5.4); the decisive gain comes from the LLM (§5.2); and the learned features transfer across time (§5.5).

5.1 Corrective discovery improves frozen and fine-tuned backbones

Figure 2a summarizes the main comparison across six datasets and six backbone families; the full per-cell grid with external baselines is in App. Ta-

ble S12. Under the shared harness of §4, both CRAFT variants outperform each of the three dedicated feature-engineering systems, with per-baseline paired Wilcoxon tests significant at $p < 0.01$. The result holds both when aggregating by dataset and when aggregating by backbone (Figure 2a and Table 1), showing that corrective discovery is not a single-dataset artifact.

The gain is not a feature-budget artifact. The same-budget sweep rules the explanation that CRAFT wins simply because it is allowed to propose more candidates. In Fig. 2b (per-budget win rates and mean differences in App. Table S11), CRAFT beats the best external feature-engineering system at every feature budget K . Even at $K=5$, CRAFT exceeds the best external system at its own uncapped budget, and the margin saturates by moderate budgets rather than appearing only in the “All” setting. The external systems’ curve is comparatively flat, indicating that their additional candidates do not translate into comparable residual-correction gains under the shared gate and corrector. Thus the main result reflects feature quality, not merely feature count.

Fine-tuning is not a substitute for residual correction. We also compare against a uniform head-only fine-tune, FT, applied identically across backbones. Fine-tuning improves the frozen forecasts, but it does not remove the structured residuals that CRAFT exploits. Correcting the zero-shot backbone beats fine-tuning alone on most cells, and applying CRAFT on top of the fine-tuned backbone still yields additional lift over FT (Fig. 2a, top

Table 1: **Cross-LLM robustness.** Per-backbone summary under both LLM backends. Both variants improve every backbone, most on the weak ones, so the backbone-strength picture is LLM-agnostic; dataset-level increments still depend on the backend (§5.2). RAW is the backbone’s mean zero-shot wMAPE $\times 100$ over the 6 datasets. **Lift%** is the mean reduction vs. RAW; **wins** counts datasets (of 6) where the variant matches or beats the best external.

Backbone	RAW	GPT-5.2				DeepSeek-3.4-pro			
		CRAFTER		CRAFTER [†]		CRAFTER		CRAFTER [†]	
		Lift%	wins	Lift%	wins	Lift%	wins	Lift%	wins
Timer	42.7	+25.7	5/6	+25.8	5/6	+23.6	3/6	+29.7	4/6
Chronos	31.7	+8.5	4/6	+11.4	5/6	+9.6	4/6	+9.5	3/6
Moirai	43.9	+27.2	5/6	+26.8	5/6	+11.5	2/6	+27.9	5/6
Toto	35.4	+14.8	4/6	+16.6	4/6	+11.0	4/6	+15.0	6/6
Chronos-2	31.3	+6.4	4/6	+7.4	4/6	+5.8	3/6	+11.9	5/6
Moirai-2	34.0	+12.5	3/6	+12.8	3/6	+12.3	4/6	+16.7	5/6

vs. bottom; the full per-cell zero-shot-versus-fine-tuned grid is App. Table S1). The same conclusion holds for the stronger full-parameter in-domain fine-tune of the covariate-capable Moirai models, reported in App. Table S2. These results place CRAFTER as a deployment-time correction layer rather than a replacement for backbone adaptation: it improves a frozen model, and it remains useful after fine-tuning when exploitable residual structure remains.

5.2 Search matters through coupling, not only standalone lift

We next discuss where the gain in §5.1 comes from. The source-blind gate in the pipeline serve as an attribution device: external feature-engineering systems, compositional search, and LLM-generated candidates are all filtered by the same validation rule and passed to the same corrector.

Search defines the enumerable feature space.

The compositional search adds little standalone lift on these public benchmarks (the noLLM column of Table S12; a component-level with/without-LLM comparison is App. Table S5), but this null result is informative rather than incidental. It shows what can be recovered by syntactic enumeration over the observed schema. When this search and the external feature banks fail to close the residual gap, but named LLM features do, the missing signal is not merely another lag, rolling aggregate, or low-order interaction. It is a semantic feature that must be named in a form aligned with the backbone’s residual. The search arm therefore acts as a control for the LLM comparison: it measures the enumerable floor, so the remaining gain is identified as

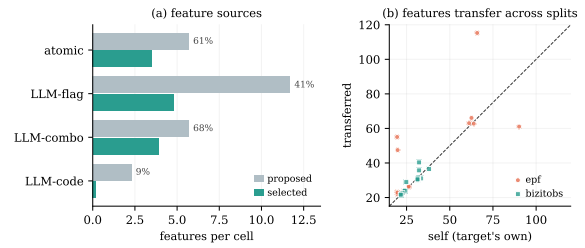


Figure 3: **Feature provenance and transfer.** (a) Features proposed vs. selected by source, with survival rate. LLM kinds supply about three-quarters of the selected set, and LLM-code is almost always pruned. (b) Features learned on one split and replayed on a later one track the target split’s own features along the diagonal. The off-diagonal points are the Moirai shift corner, mostly single-series EPF (§5.5; App. L).

a semantic feature gap rather than assumed to be one.

The clearest case is EPF. External banks and syntactic search add little beyond the covariate-only corrector, while CRAFTER removes much more of the frozen error using named forecast-covariate features such as `renewable_penetration` (the EPF rows of Table S12; representative accepted features in App. K). This is the regime corrective discovery is designed for: the relevant information is present in the covariates, but it does not become useful until it is expressed as a feature that matches the backbone’s failure mode. Similar, though smaller, effects appear on multi-series retail panels, where domain-named interactions generalize across related store–product series.

Coupling turns search from a control into a mechanism. The coupled variant, CRAFTER[†], enables the two cross-feeding generators in §3.2: search results summarize which enumerable candidates survived the gate, and that feedback is exposed to the LLM in later proposal rounds. Search alone adds little net lift, yet feeding its accepted and rejected candidates back to the LLM makes CRAFTER[†] stronger on balance: it improves on CRAFTER on more cells, gives per-backbone lift at least as large on all but one backbone, and has its clearest gains on EPF (per-backbone lift in Table 1; per-cell in Table S12). Since the search contributes little alone, that gain is the cross-feeding steering the LLM toward features it would not otherwise propose. Its cost is variance, especially on volatile single-series cells such as EPF, where a small change in the feature set can move a cell in either direction (seed versus split variance in App. D).

We therefore report CRAFT_{ER} and CRAFT_{ER}[†] as two endpoints of an explore–exploit trade-off.

Feature-level evidence supports the same mechanism. The provenance analysis in Fig. 3a shows that most selected features come from LLM-generated semantic sources, while atomic search features survive at a comparable rate but are limited by the smaller enumerable pool (per-source and per-dataset counts in App. J). This matches the performance attribution: the LLM supplies most of the selected feature mass, while search marks the reliable enumerable candidates and provides feedback for coupling. Code-generated features are rarely selected, probably constrained by limited program synthesis. Instead, CRAFT_{ER} works by combining three roles: a corrector that tests whether a feature actually reduces residual error, a search process that establishes and feeds back the enumerable space, and an LLM that names semantic features outside that space.

5.3 When do corrective features help?

The central condition is residual headroom, and App. Fig. S1 is its most direct support: plotting every backbone–dataset cell’s corrected gain against the backbone’s own RAW error, CRAFT_{ER}’s lift concentrates where the raw error is large and collapses to zero where the backbone has already saturated the series. Corrective features help when the frozen backbone leaves structured errors that external signals can explain; they are neutral or harmful otherwise. This condition is stronger than backbone identity, release order, or feature budget.

Headroom, not backbone identity, determines the gain. The clearest evidence comes from comparing the same dataset across backbone families. On BIZITOBS, correction gives large lift when the raw backbone is weak, as with Timer and Moirai, but becomes neutral or negative when the raw forecast already saturates the series, as with Chronos and Moirai-2.0. The gain is governed by how much residual structure the backbone leaves behind, not by which foundation model produced the forecast. This also explains why newer or stronger backbones can still benefit on some datasets, such as ROSSMANN and EPF, while older or weaker backbones do not automatically improve everywhere.

The useful feature source depends on the kind of residual structure. When the remaining error is tied to semantic forecast covariates, named features matter most. This is the case on EPF, where external feature banks and syntactic search add lit-

tle beyond the covariate-only corrector, but LLM-named features close much more of the frozen error (the EPF rows of Table S12, whose selected set is LLM-dominated in App. Table S7). When the available covariates are already directly informative, as on the retail panels, the covariate-only corrector already captures a large share of the gain and additional searched features are often redundant (the ROSSMANN and FAVORITA rows of Table S12). When the residual contains no stable structure, no feature source reliably helps and every column collapses onto RAW (the saturated M5 rows, and the flat right-hand region of App. Fig. S1). The question is therefore not whether one source is globally best, but which source matches the residual left by a particular backbone on a particular dataset.

Budget amplifies headroom but does not create it. The same-budget sweep gives the feature-budget axis of the characterization (Fig. 2b; App. Table S11). Increasing K widens CRAFT_{ER}’s margin in cells where residual headroom exists, and the gains saturate by moderate budgets. But a larger budget does not turn saturated cells into wins: if the backbone has already absorbed the available structure, more candidate features mostly add variance and selection risk. Budget is therefore an amplifier, not a switch. It helps exploit residual structure when that structure is present, but cannot manufacture structure where the residual is already close to noise. The practical guidance is therefore simple: deploy corrective feature discovery when validation residuals remain structured, especially in weak-backbone or semantic-covariate regimes; treat feature budget as a way to scale gains in those regimes; and avoid forcing correction in saturated cells where no feature source passes the validation gate reliably.

5.4 Robustness to backend and input path

The preceding sections use a fixed LLM backend and route all auxiliary information through the corrector. We check that the method and the headroom characterization survive two deployment choices: which LLM proposes features, and where covariates or multivariate inputs are injected.

Changing the LLM backend preserves the characterization. Repeating the full grid with DeepSeek-3.4-pro yields the same qualitative pattern as the main backend: both CRAFT_{ER} variants improve every backbone on average, and the gains remain largest where the raw backbone leaves more residual headroom (Table 1; per-cell results in

App. Table S13). Thus the conclusion of §5.3 is not an artifact of one LLM. The backend does affect individual cells, especially volatile single-series cases such as EPF, so we do not claim backend-invariant feature sets or identical per-dataset increments. The robust claim is weaker and more useful: changing the LLM changes which named features are found, but it does not change the deployment rule that correction helps when exploitable residual structure remains.

Covariate-capable backbones do not replace the corrector. A second deployment choice is whether to feed covariates directly to a backbone that accepts them, or to keep the backbone forecast-only and route covariates to the residual corrector. The direct route does not dominate. Toto ingests covariates harmlessly, with little change in RAW, while Moirai-2.0 can degrade substantially when given the same inputs directly (App. Table S3). In contrast, the correction layer is stable across this choice: on the retail panels, CRAFT reaches similar corrected accuracy whether or not the covariates were first exposed to the backbone, and it can recover even when covariate ingestion degrades the raw forecast. This supports the design choice in §3.1: keep the backbone fixed and route auxiliary signals to a learnable residual corrector.

5.5 Learned features transfer across time

A corrective feature is useful only if it captures persistent structure rather than a single-window artifact. We test this by *replaying* the features accepted on one split onto a later split, with the LLM and search turned off, so only the feature set’s origin changes; the target split’s own features serve as an upper bound, and a covariate-only corrector as a lower bound. Across two datasets, five backbones, and every ordered split pair, transferred features match the target’s own features at the median and stay within a few percent on most cells (Fig. 3b; App. Table S9). Performance decays only mildly with split distance, while catastrophic failures concentrate in the Moirai shift corner, especially the volatile single-series EPF. Despite low overlap in accepted feature names across splits, performance transfers because different runs often discover functionally equivalent features. Corrective feature discovery therefore recovers reusable residual structure, not per-window noise.

6 Conclusion

We studied corrective feature discovery: correcting a frozen black-box forecaster by mining interpretable features of its residual instead of retraining it. CRAFT mines the residual with a compositional search and a language model, and admits their candidates through one validation-grounded gate that is blind to a feature’s origin. The shared gate turns two generators into a single method and lets any external feature-engineering system run through the identical pipeline, so CRAFT also serves as an instrument that attributes a change in the forecast to the feature source alone. Correction pays in proportion to the residual’s exploitable structure: large on weak backbones with informative forecast covariates, where LLM-named features are the differentiator, and neutral once a strong backbone already fits the series—we report the saturated cells alongside the wins. The finding holds across six backbones and two LLM backends, is not bought with a larger feature budget, and survives fine-tuning. Because features learned on one window transfer to the next on the datasets we test (§5.5), a natural next step is a lightweight feature-memory that carries mined features forward to amortize the search and warm-start the corrector, adapting as a backbone’s errors evolve across windows.

References

- Nikhil Abhyankar, Parshin Shojaei, and Chandan K. Reddy. 2025. Llm-fe: Automated feature engineering for tabular data with llms as evolutionary optimizers. *arXiv preprint arXiv:2503.14434*.
- Abdul Fatir Ansari, Oleksandr Shchur, Jari Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, Mononito Goswami, Shubham Kapoor, Danielle C. Maddix, Pablo Guerron, Tony Hu, Junming Yin, Nick Erickson, Prateek Mutalik Desai, Hao Wang, and 4 others. 2025. *Chronos-2: From univariate to universal forecasting*. *Preprint*, arXiv:2510.15821.
- Abdul Fatir Ansari, Lorenzo Stella, Ali Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Bernie Wang. 2024. *Chronos: Learning the language of time series*. *Transactions on Machine Learning Research*. Expert Certification.

- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256.
- Vitor Cerqueira, Nuno Moniz, and Carlos Soares. 2020. Vest: Automatic feature engineering for forecasting. *arXiv preprint arXiv:2010.07137*.
- Hamza Cherkaoui, Malik Tiomoko, Giuseppe Paolo, Yili Zhang, Meng Yu, Keli Zhang, and Hafiz Tiomoko Ali. 2025. Post-training corrections for improved time-series forecasting. *arXiv preprint arXiv:2505.15354*.
- Maximilian Christ, Nils Braun, Julius Neuffer, and Andreas W. Kempa-Liehr. 2018. Time series FeatuRE extraction on basis of scalable hypothesis tests (ts-fresh – a Python package). *Neurocomputing*, 307:72–77.
- Ben Cohen, Emaad Khwaja, Kan Wang, Charles Masson, Elise Ramé, Youssef Doubli, and Othmane Abou-Amal. 2024. Toto: Time series optimized transformer for observability. *arXiv preprint arXiv:2407.07874*.
- Pedro Miguel Pinto Costa. 2021. Autofits: Automated feature engineering for irregular time-series. Master’s thesis, Universidade do Porto.
- Xilin Dai, Yiding Liu, Hongjie Xia, Yifan Hu, Zewei Dong, Jiang-Ming Yang, and Qiang Xu. 2026a. Learning the context of errors: Black-box online adaptation of time series foundation models. *arXiv preprint arXiv:2606.14222*.
- Ziming Dai, Dabiao Ma, Jinle Tong, Mengyuan Han, Jian Yang, Hongtao Liu, Haojun Fei, and Qing Yang. 2026b. NSR-Boost: A neuro-symbolic residual boosting framework for industrial legacy models. *arXiv preprint arXiv:2601.10457*.
- Langzhou He, Junyou Zhu, Fangxin Wang, Junhua Liu, Haoyan Xu, Yue Zhao, Philip S. Yu, and Qitian Wu. 2025. [Can molecular foundation models know what they don’t know? a simple remedy with preference optimization](#). *Preprint*, arXiv:2509.25509.
- Noah Hollmann, Samuel Müller, and Frank Hutter. 2023. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *arXiv preprint arXiv:2305.03403*.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. [Parameter-efficient transfer learning for nlp](#). *Preprint*, arXiv:1902.00751.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. [Lora: Low-rank adaptation of large language models](#). *Preprint*, arXiv:2106.09685.
- Yiran Huang, Yexu Zhou, Michael Hefenbrock, Till Riedel, Likun Fang, and Michael Beigl. 2022. Automatic feature engineering through Monte Carlo tree search. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, volume 13715 of *Lecture Notes in Computer Science*. Springer.
- Kaggle. 2015. Rossmann store sales. Kaggle competition, <https://www.kaggle.com/c/rossmann-store-sales>.
- Kaggle. 2017. Corporación favorita grocery sales forecasting. Kaggle competition, <https://www.kaggle.com/c/favorita-grocery-sales-forecasting>.
- Kaggle. 2024. Rohlik sales forecasting challenge. Kaggle competition, <https://www.kaggle.com/competitions/rohlik-sales-forecasting-challenge>.
- Udayan Khurana, Horst Samulowitz, and Deepak Turaga. 2018. Feature engineering for predictive modeling using reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.
- Levente Kocsis and Csaba Szepesvári. 2006. Bandit based monte-carlo planning. In *Machine Learning: ECML 2006*, pages 282–293. Springer.
- Jesus Lago, Grzegorz Marcjasz, Bart De Schutter, and Rafał Weron. 2021. [Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark](#). *Applied Energy*, 293:116983.
- Daojun Liang, Qi Li, Yinglong Wang, Jing Chen, Hu Zhang, Xiaoxiao Cui, Qizheng Wang, and Shuo Li. 2026. The forecast after the forecast: A post-processing shift in time series. *arXiv preprint arXiv:2601.20280*.
- Chenghao Liu, Taha Aksu, Juncheng Liu, Xu Liu, Han-shu Yan, Quang Pham, Silvio Savarese, Doyen Sahoo, Caiming Xiong, and Junnan Li. 2025a. Moirai 2.0: When less is more for time series forecasting. *arXiv preprint arXiv:2511.11698*.
- Fengze Liu, Haoyu Wang, Joonhyuk Cho, Dan Roth, and Andrew W. Lo. 2025b. AutoCT: Automating interpretable clinical trial prediction with LLM agents. *arXiv preprint arXiv:2506.04293*.
- Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. 2024. Timer: generative pre-trained transformers are large time series models. In *Proceedings of the 41st International Conference on Machine Learning*, pages 32369–32399.
- Alfredo Lopez and Florian Sobieczky. 2024. Surrogate modeling for explainable predictive time series corrections. *arXiv preprint arXiv:2412.19897*.

- Sisuo Lyu, Siru Zhong, Tiegang Chen, Weilin Ruan, Qingxiang Liu, Taiqiang Lv, Qingsong Wen, Raymond Chi-Wing Wong, and Yuxuan Liang. 2026. TS-Memory: Plug-and-play memory for time series foundation models. *arXiv preprint arXiv:2602.11550*.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2022. [M5 accuracy competition: Results, findings, and conclusions](#). *International Journal of Forecasting*, 38(4):1346–1364.
- Andrew Murray, Danial Dervovic, and Michael Cashmore. 2025. Elate: Evolutionary language model for automated time-series engineering. *arXiv preprint arXiv:2508.14667*.
- Jaehyun Nam, Kyuyoung Kim, Seunghyuk Oh, Jihoon Tack, Jaehyung Kim, and Jinwoo Shin. 2024. Optimized feature generation for tabular data via llms with decision tree reasoning. *arXiv preprint arXiv:2406.08527*.
- Santosh Palaskar, Vijay Ekambaram, Arindam Jati, Neelamadhav Gantayat, Avirup Saha, Seema Nagar, Nam H. Nguyen, Pankaj Dayama, Renuka Singhgatta, Prateeti Mohapatra, Harshit Kumar, Jayant Kalagnanam, Nandyala Hemachandra, and Narayan Rangaraj. 2024. [Automixer for improved multivariate time-series forecasting on business and it observability data](#). *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(21):22962–22968.
- Oleksandr Shchur, Ali Caner Turkmen, Nick Erickson, Huibin Shen, Alexander Shirkov, Tony Hu, and Bernie Wang. 2023. AutoGluon-TimeSeries: AutoML for probabilistic time series forecasting. In *Proceedings of the Second International Conference on Automated Machine Learning (AutoML)*.
- Zhendong Wang, Ioanna Miliou, Isak Samsten, and Panagiotis Papapetrou. 2023. Counterfactual explanations for time series forecasting. *arXiv preprint arXiv:2310.08137*.
- Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- G. Peter Zhang. 2003. Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing*, 50:159–175.
- Tianping Zhang, Zheyu Zhang, Zhiyuan Fan, Haoyan Luo, Fengyuan Liu, Qian Liu, Wei Cao, and Jian Li. 2022. OpenFE: Automated feature generation with expert-level performance. *arXiv preprint arXiv:2211.12507*.

Appendix

Tables, figures, algorithms and equations below carry an S prefix (Table S1, Fig. S1, Alg. S1, Eq. (S1)); plain references (§3.2, Fig. 2a, Table 1) point to the main paper above.

A Correction Target and Metric

For each series and forecast origin we observe a history $x_{1:T}$, a frozen backbone forecast $\hat{y}_{1:H}$ over horizon H , and (at training time) the truth $y_{1:H}$. CRAFTER predicts a per-series multiplicative correction factor

$$\begin{aligned}\tilde{y}_{1:H} &= f \cdot \hat{y}_{1:H}, \\ f &= \text{clip}\left(\frac{\sum_h y_h}{\sum_h \hat{y}_h}, \ell, u\right), \quad [\ell, u] = [0.3, 3.0],\end{aligned}\quad (\text{S1})$$

with the clip admitting genuine multiplicative structure (e.g., promotions, price changes) while rejecting heavy-tailed ratios from near-zero denominators; an additive-offset target $f^+ = \sum_h y_h - \sum_h \hat{y}_h$ is also supported and selected against within the corrector set (§3.4). We report the weighted mean absolute percentage error of the horizon total,

$$wMAPE = \frac{\sum_i \left| \sum_h \tilde{y}_{i,h} - \sum_h y_{i,h} \right|}{\sum_i \left| \sum_h y_{i,h} \right|}. \quad (\text{S2})$$

We score the horizon *total* because it is the quantity deployment consumes—replenishment, procurement, and capacity decisions are driven by aggregate demand over the lead time—and the quantity the correction factor of Eq. (S1) acts on. The comparison stays fair: every method, the frozen backbone included, is scored on the same total.

B The CRAFTER Round

Alg. S1 gives one round of the loop sketched in §3.1. A run repeats it for R rounds, threading the accepted-feature set $\mathcal{F}$, the search arm statistics Θ , and the channel pool $\mathcal{C}$ across rounds. The two generators of §3.2 appear as *Exploration* (compositional UCB search, §3.2) and *Reasoning* (the LLM roles, §3.2); both feed the single source-blind gate of §3.3 (line 6), which keeps only candidates whose Spearman correlation with the *unexplained* validation residual exceeds τ , that are not collinear with an accepted feature, and that fit the per-family and per-round budgets. After refitting, a per-tier permutation step prunes the lowest-value features by Δ validation-wMAPE (line 8), and the

kept/trimmed outcome plus the realized validation gain is credited back to the search arms as UCB feedback. The cross-round coupling shown—LLM-named base columns entering $\mathcal{C}$ so the search can compose them (line 7)—is one of the two channels CRAFTER[†] enables (§3.2); it is off in the decoupled CRAFTER.

C Feature Grammar

A corrective feature is a structured specification: a typed record that compiles deterministically to one numeric column over items. Specifications take one of three types, and every field is drawn from a fixed vocabulary, so the space is bounded, searchable, and canonicalizable. The atomic search emits combo records; the LLM emits combo, flag, and code records, which the rest of the paper names **LLM-combo**, **LLM-flag**, and **LLM-code**.

- **combo** — a pairwise expression $\text{POST}(\tau_a(c_a) \oplus \tau_b(c_b))$, where c_a, c_b are input channels (base features or covariates), $\tau_\bullet$ are unary transforms, $\oplus$ is a binary operator, and POST is an optional output transform with an optional clip (a single transformed channel when $c_b, \oplus$ are omitted).
- **flag** — a 0/1 indicator formed by combining a small set of thresholded conditions $\{(c, \text{op}, v)\}$ with all/any.
- **code** — a short Python function over the target history and the backbone forecast, restricted by an abstract-syntax-tree allow-list to `numpy`, `math`, and `scipy`.`{signal,stats,fft,special}` with no I/O, `exec`, or `eval`.

Vocabulary. The search ranges over $B=8$ binary operators $\oplus \in \{\text{add, sub, mul, div, abs_diff, min, max, safe_div}\}$ and $T=15$ unary transforms $\tau \in \{\text{null, neg, abs, log, log1p, signlog, sqrt, square, exp, sigmoid, tanh, softplus, zscore, winsorize, ema}\}$; flag conditions draw from the operators $\{<, \leq, >, \geq, =, \neq, \text{between, in, not_in}\}$. The executor additionally admits a few rarer transforms and operators (e.g., `geo_mean`, quantile summaries); these are reachable by the LLM but outside the atomic search’s UCB vocabulary.

Feature families. Each channel belongs to one mechanism *family*, the unit L1 routes over (§3.2). Representative families and example

channels: level (mean_sales, zero_ratio); trend (trend_ratio, trend_consistency); seasonality (autocorr7, seasonal_strength); volatility (cv, outlier_rate); shift (regime_shift, shift_magnitude); anomaly (spike_ratio, consecutive_zeros).

Example. The JSON record {type:combo, feat_a:cv, feat_b:mean_sales, op:div, transform_a:log, transform_b:null} denotes $\log(\text{cv})/\text{mean_sales}$ and is named accordingly. A *canonical key* maps equivalent records to one identifier: the display name and all book-keeping fields (source, family, scores) are dropped, the empty/null/identity transforms collapse to a single sentinel, numeric thresholds and clips are rounded, and flag conditions are sorted, so the key is an order- and rename-invariant hash of the remaining grammar fields. It drives cross-round deduplication and a rejection blacklist that suppresses any candidate already tried and refused.

D Corrector and Hyperparameters

The corrector is a histogram gradient-boosted regressor (200 trees, max depth 4, learning rate 0.1, min-samples-leaf 20). The corrector set comprises {NONE, additive, multiplicative-factor} correctors, the best selected by validation wMAPE; the multiplicative-factor head is the strongest on weak-backbone $\times$ forecast-covariate cells, where a per-row learned factor is more flexible than per-horizon offsets. We fix its clip to [0.3, 3.0] rather than a dataset-adaptive bound: an adaptive bound is occasionally tighter on clean retail but blows up on weak-backbone $\times$ LLM cells, so we trade a little adaptivity to remove that failure mode. Acceptance gate: a candidate is kept iff $|\rho_{\text{Spearman}}(\text{feature}, \text{unexplained residual})| \geq \tau$, pairwise $|\rho| \leq 0.75$, with at most two features per family and ten per round. The threshold is a single global constant $\tau=0.05$, fixed across all datasets rather than tuned per series, so there is no per-dataset grid. Search defaults: a four-layer conditioned UCB1 tree with exploration constant $\kappa=1.4$, $R=3$ rounds, and $K=4$ planner directions; the selection, update, and reward rules follow.

Bandit selection and update. The compositional search is a Monte-Carlo tree of UCB1 bandits with one arm per choice at each layer (family, then arity, then channels, then operator). Every arm a stores a visit count n_a and a running mean reward $\hat{\mu}_a$; each L1 family arm keeps one $(n_a, \hat{\mu}_a)$ pair per

arity, so univariate and bivariate evidence never mix. At a node whose children are $\mathcal{A}$ with total visits $N = \sum_{a' \in \mathcal{A}} n_{a'}$, the descent selects the arm that maximizes the UCB1 score,

$$a^* = \arg \max_{a \in \mathcal{A}} \hat{\mu}_a + \kappa \sqrt{\frac{\ln N}{n_a}}, \quad (\text{S3})$$

visiting any never-tried arm ($n_a=0$) first. Once an arm’s candidate has been scored, or a round has finished, the arm is credited with a reward r and updated by the incremental sample mean

$$n_a \leftarrow n_a + 1, \quad \hat{\mu}_a \leftarrow \hat{\mu}_a + \frac{1}{n_a} (r - \hat{\mu}_a), \quad (\text{S4})$$

applied to every arm on the selected root-to-leaf path; a decayed copy is also propagated to sibling contexts that share the family or base channel. The reward combines two signals. A fast per-candidate proxy is available already at scoring time,

$$r_{\text{proxy}} = \alpha |\rho| + \beta \mathbb{I}[\text{accepted}], \quad \alpha=0.4, \beta=0.3, \quad (\text{S5})$$

where ρ is the Spearman correlation between the candidate and the still-unexplained validation residual and $\mathbb{I}[\text{accepted}]$ marks passing the acceptance gate above. After the corrector is refit at each round’s end, a realized signal supplements the proxy: every surviving feature is credited in proportion to its gradient-boosting importance, plus a $\gamma=0.3$ round-level term on the round’s validation-wMAPE improvement. The proxy steers exploration cheaply within a round; the realized signal ties the search to the deployed corrector, so a feature is ultimately worth what keeping it actually buys.

Why gradient-boosted trees. The residual is nonlinear and regime-dependent, concentrating in promotions, regime shifts, and rare events, so a linear corrector cannot fit it. Gradient-boosted trees have the capacity and keep the correction auditable: a per-feature importance names *which* mechanism drives a backbone’s residual. Read across backbones, the corrector *diagnoses* each model’s blind spots, naming what a given backbone systematically misses (§5.2).

Protocol and compute. All backbone forecasts are computed once on NVIDIA A100/A40 GPUs and cached; because random seeds do not change a frozen backbone’s forecast, the entire multi-seed grid (seeds 42–44 $\times$ three rolling splits per cell)

reuses the warm cache with *no* GPU recomputation, and the residual-correction loop itself runs on CPU. The stack is Python 3.12, with PyTorch 2.x for the backbone runners and LightGBM 4.x for the corrector. A full single-seed sweep of the grid completes in ~ 30 minutes on warm caches—orders of magnitude cheaper than per-dataset backbone fine-tuning, which requires a GPU training run for every (dataset, backbone) pair. We report last-round ($R=3$) test wMAPE, averaged over the three splits and three seeds, and decompose variance into seed (optimizer/LLM-sampling) and split (temporal-drift) components; split variance dominates. Quantified per cell on the DeepSeek grid (wMAPE $\times 100$): the median seed-std of CRAFT is 0.49 against a median split-std of ~ 3.2 ; the frozen backbone is seed-deterministic, the no-LLM and TSFRESH columns sit near 0.02, and the LLM methods’ seed noise concentrates on EPF.

Per-arity statistics and the univariate transform.

Each L1 family arm stores two independent (count, mean) pairs, one per arity, so univariate and bivariate evidence never mix. We do not search the univariate transform: with only $T=15$ options and that branch serving mainly to surface new channels for later composition, a bandit there would chiefly drain samples from the bivariate branch, where the BT^2 operator-triple choice carries the reward. Because the bivariate branch is sampled far more often, its visit count grows much faster; we periodically rebalance the two arities’ counts so the exploration bonus $\kappa\sqrt{\ln N/n}$ does not lock onto the under-sampled univariate arity.

E External-System Adaptation

The three external systems run *inside* the CRAFT harness: the same cached backbone residual, the same base and covariate columns, the same gradient-boosted corrector and corrector-set selection, and the same validation top- K feature selector. Only the candidate generator is swapped, so the comparison isolates feature generation (§4). Each generator is kept as close to its published form as the forecasting setting allows.

- **TSFRESH** extracts its standard statistical bank over each series history through the library’s own extractor (~ 30 statistics per series), from which the shared selector keeps the top K .
- **CAAFE** targets classification, so its LLM iteration loop’s validation-accuracy acceptance

test runs on the residual ratio discretized into three bins (over-, well-, and under-predicted); the accepted features themselves remain continuous columns for the shared corrector.

- **LLM-FE** runs its evolutionary loop, with LLM-proposed mutations and crossovers over a population of feature expressions and fitness the validation correlation against the same residual target.

The two LLM-based systems use the same backend, per-run seed, and feature budget as CRAFT. We do not run the systems’ native end-to-end pipelines, because they target tabular prediction with their own downstream model: a native run would change the corrector, the target, and the metric at once, and a difference in the final forecast could then no longer be attributed to the feature source. This inside-the-harness design is the point of the instrument (§3.3); its cost is that we measure each system as a feature generator for residual correction, not its native end-to-end performance.

F Fine-tuning baseline

The FT baseline in Table S12 applies a *uniform head-only* fine-tune to every backbone (training the output head with the backbone frozen), early-stopping on validation wMAPE with the test window untouched, on splits identical to CRAFT; a constant recipe keeps the comparison fair across backbones. As a stronger check, the two Moirai backbones, which ship an official recipe, additionally receive a full-parameter in-domain fine-tune (Table S2): it reuses the official uni2ts MOIRAFINETUNE module and loss but masks the last H patches (eval-aligned) rather than at random, selects checkpoints on validation wMAPE rather than NLL, and trains in fp32 with gradient clipping and warmup. The comprehensive grid (Table S1; all six backbones $\times$ six datasets, both backbone branches $\times$ four correction gradients) reports this head-only FT next to each correction level. The grids are single-seed (42), split-averaged, so absolute values differ slightly from the 3-seed main table, most visibly on the volatile single-series EPF; the before/after comparison of Fig. 2a (top vs. bottom) uses both branches within the same seed and is unaffected. Fine-tuning requires a GPU training run for every (dataset, backbone) pair, orders of magnitude more costly than the GPU-free correction loop (App. D).

Table S1: **Fine-tuning does not remove the gain (full grid).** Test wMAPE $\times 100$ (lower is better) for the four correction gradients—no correction (RAW zero-shot / FT fine-tuned), covariate-only corrector (COV), CRAFTER, and CRAFTER[†]—on the zero-shot and the fine-tuned backbone, over all 6 backbones $\times$ 6 datasets (CHR₂ is covariate-aware Chronos-2, single-seed; its covariate-only corrector was not re-run on the fine-tuned branch, so the two COV columns carry the same value). Fine-tuning is the uniform head-only probe (full-parameter Moirai in Table S2). On the non-saturated datasets (EPF, ROSSMANN, ROHLIK) the ordering CRAFTER < COV < no-correction holds on *both* branches; M5 is saturated and all gradients coincide. The CRAFTER and CRAFTER[†] columns (gray) are highlighted in both branches. **Bold** marks the best gradient within each branch (a CRAFTER variant when it ties). Single seed (42), split-averaged.

Dataset	BB	Zero-shot backbone				Fine-tuned backbone (head-only)			
		RAW	COV	CRAFTER	CRAFTER [†]	FT	COV	CRAFTER	CRAFTER [†]
epf	tim	53.7	46.2	30.2	30.2	56.9	43.5	31.1	31.2
	chr	49.9	45.4	32.0	31.7	52.0	44.2	32.6	30.2
	moi	73.2	60.7	36.8	28.4	71.2	56.4	44.5	30.2
	toto	55.8	49.8	31.6	28.8	55.8	49.8	40.4	30.7
	chr ₂	52.6	43.7	37.1	32.2	55.1	43.7	42.4	55.0
	moi ₂	55.0	41.6	38.6	50.7	55.0	41.8	42.1	29.7
rossmann	tim	39.5	16.7	13.5	13.5	39.5	16.8	13.7	13.4
	chr	20.6	11.9	11.3	11.3	21.3	14.0	13.4	13.2
	moi	30.0	24.7	13.4	13.1	29.8	24.7	13.0	13.1
	toto	22.2	14.5	12.5	12.2	22.2	14.4	12.2	12.2
	chr ₂	20.4	15.9	13.1	12.8	21.1	15.9	12.5	14.0
	moi ₂	21.4	12.6	11.7	11.7	21.4	12.6	12.1	11.7
rohlik	tim	36.1	33.4	28.6	28.6	30.3	27.2	26.7	26.7
	chr	27.4	24.7	24.4	24.4	27.2	24.5	24.3	24.3
	moi	36.2	33.5	28.5	28.4	31.2	28.3	27.6	27.6
	toto	28.2	25.1	24.8	24.9	28.2	25.3	25.1	25.2
	chr ₂	26.6	23.8	23.6	23.6	27.1	23.8	24.0	23.9
	moi ₂	34.9	32.2	28.0	28.0	29.2	26.3	25.4	25.4
bizitobs	tim	49.5	42.4	36.7	41.9	51.9	41.8	42.3	34.3
	chr	24.6	25.7	30.1	24.6	25.0	25.6	26.1	25.9
	moi	51.5	43.5	39.1	39.3	50.9	42.3	40.2	36.8
	toto	39.0	40.9	37.6	35.3	39.0	40.9	35.2	37.2
	chr ₂	23.6	31.0	31.1	31.4	25.3	31.0	30.6	30.4
	moi ₂	27.1	27.7	27.3	26.3	27.1	27.7	26.6	31.1
favorita	tim	24.9	21.8	21.4	21.7	23.2	21.6	20.5	20.5
	chr	17.5	17.6	17.5	17.6	17.4	17.6	17.1	17.4
	moi	20.6	19.8	19.7	19.7	20.6	19.8	19.9	19.9
	toto	17.4	17.7	17.5	17.4	17.4	17.7	17.4	17.5
	chr ₂	14.4	15.2	15.3	15.9	14.6	15.2	15.7	15.4
	moi ₂	16.5	16.6	16.5	16.5	16.5	16.6	16.6	16.5
m5	tim	52.5	51.0	51.1	51.2	51.4	50.8	50.9	50.9
	chr	50.3	50.3	50.1	50.0	49.9	49.9	50.0	49.9
	moi	52.2	52.1	51.2	50.9	52.2	52.1	51.0	51.0
	toto	49.7	50.1	49.9	50.0	49.7	50.1	49.9	50.0
	chr ₂	50.4	51.2	50.2	50.3	49.5	51.2	50.3	50.3
	moi ₂	49.0	49.3	49.5	49.5	49.0	49.3	49.3	49.5

Table S2: **Full-parameter Moirai stress test.** The four correction gradients on a *fully* fine-tuned Moirai backbone—no correction (FT), covariate-only corrector (COV), CRAFT, and CRAFT[†]—alongside the zero-shot branch, for the two covariate-capable Moirai models (test wMAPE $\times 100$, lower is better). CRAFT still helps on 6 of 8 fine-tuned cells; fine-tuning rarely beats correcting the frozen backbone. The CRAFT/CRAFT[†] columns (gray) are highlighted; **bold** marks a corrector that beats both no-correction baselines in its branch (RAW/COV on the left, FT/COV on the right), the Table S12 rule. Exceptions: FAVORITA, BIZITOBS under Moirai-2.0. Single seed (42), split-averaged.

Dataset	BB	Zero-shot backbone				Fine-tuned backbone (full-param)			
		RAW	COV	CRAFT	CRAFT [†]	FT	COV	CRAFT	CRAFT [†]
epf	moi	73.2	60.8	36.7	28.4	60.8	55.5	37.8	44.4
	moi ₂	55.0	41.6	38.6	45.8	55.0	41.6	32.2	29.0
rossmann	moi	30.0	26.2	13.2	13.0	26.1	24.9	17.2	16.4
	moi ₂	21.4	12.6	11.8	12.0	21.4	12.6	11.9	11.6
bizitobs	moi	51.5	43.5	38.5	39.5	55.1	53.5	40.7	38.8
	moi ₂	27.1	30.1	31.0	30.5	27.1	30.1	29.6	28.9
favorita	moi	20.6	20.3	20.1	20.4	25.4	23.3	22.5	22.4
	moi ₂	16.5	16.6	16.5	16.5	16.5	16.6	17.9	17.8

G Covariate routing and native multivariate forecasting

These two ablations support §5.4: the corrector’s gain is decoupled from how a covariate-capable or multivariate-capable backbone uses its richer input path.

Covariates in the backbone vs. the corrector.

Table S3 feeds known covariates to the frozen backbone (RAW+cov) instead of routing them to the corrector. Toto ingests them harmlessly ($\Delta \leq 0$ on every dataset) while Moirai-2.0 misuses them, degrading RAW by up to 45.8% (ROSSMANN); neither backbone turns covariate access into a real gain. Routing the same covariates to the corrector instead yields large gains, and CRAFT reaches comparable accuracy whether or not the backbone was given the covariates—recovering even where the cov-fed backbone degraded its own RAW (Moirai-2.0 ROSSMANN, 31.1 $\rightarrow$ 13.8 vs. 11.8 on the univariate backbone). The corrector’s gain is therefore largely independent of how, or whether, the backbone ingests covariates; the lone volatile EPF series is the one exception. Timer and Chronos are univariate and cannot ingest covariates.

Native multivariate forecasting. BIZITOBS is the one panel with several jointly forecastable channels. Table S4 compares per-channel (uni) with joint (mv) backbone forecasting on the two natively multivariate backbones. Joint forecasting improves

Toto’s any-variate RAW and badly degrades the small Moirai-2.0, but the corrector behaves identically on either forecast (the same small lift in both the uni and mv columns), so the uni/mv gap is set by the backbone, not the wrapper. Timer and Chronos are univariate; Moirai is multivariate-capable but not run here. Importantly, native multivariate forecasting changes the backbone, not the wrapper. On BIZITOBS, the only panel where several channels can be forecast jointly, native multivariate forecasting is strongly backbone-dependent: it improves Toto’s raw forecast but degrades the smaller Moirai-2.0 model (App. Table S4). The correction layer behaves similarly on either raw forecast. Thus the univariate–multivariate gap is mainly a property of the backbone’s input path, whereas CRAFT remains a black-box residual correction layer that applies after either choice.

H LLM Roles and Prompts

CRAFT calls a single LLM provider in four roles: a fast tier runs the PLANNER, a reasoning tier runs the SPECIFIER and REFLECTION, and a BASE PROPOSER invents new base columns. The legacy RERANKER is disabled—the merged candidate pool is gated deterministically by the source-blind rule of §3.3, not by an LLM. Across a run (three rounds) the LLM is queried about a dozen times (~ 4 per round): each exploration round runs a PLANNER that proposes the round’s directions, a SPECIFIER per direction, a BASE PROPOSER, and

Table S3: **Covariates belong in the corrector, not the backbone** (DeepSeek-3.4-pro backend, test wMAPE $\times 100$, three-split mean, seed 42). RAW is the univariate backbone; RAW+cov feeds covariates to the frozen backbone, with Δ the resulting change in RAW (positive = worse). CRAFTER (uni) and CRAFTER (cov) place the corrector on the univariate and the cov-fed backbone respectively (the two gray columns; **bold** = beats both RAW and RAW+cov in that row, the Table S12 rule). Feeding covariates to the backbone is at best neutral (Toto, $\Delta \leq 0$ throughout) and often harmful (Moirai-2.0), whereas routing them to the corrector yields large gains; the two CRAFTER columns match across the retail panels, so the corrector’s gain is decoupled from how the backbone routes covariates (EPF excepted). Timer and Chronos are univariate and cannot ingest covariates; Moirai covariate injection was run single-split only; BIZITOBs has no covariates. M5 is evaluated on this grid’s own series basis, so its RAW differs slightly from Table S12; compare within the table.

BB	Dataset	RAW	RAW+cov	Δ	CRAFTER (uni)	CRAFTER (cov)
Toto	epf	55.8	52.6	−5.7%	39.3	50.9
	rossmann	22.2	22.1	−0.7%	13.5	13.6
	rohlik	28.2	27.9	−0.9%	25.0	24.8
	favorita	17.4	17.4	−0.1%	17.6	17.6
	m5	50.6	49.3	−2.5%	50.6	49.4
Moirai-2	epf	55.0	54.5	−0.9%	46.5	42.9
	rossmann	21.4	31.1	+45.8%	11.8	13.8
	rohlik	34.9	35.3	+1.0%	32.3	31.9
	favorita	16.5	18.9	+14.5%	16.5	18.2
	m5	49.9	49.8	−0.2%	50.0	50.2

Table S4: **Native multivariate forecasting on BIZITOBs** (DeepSeek-3.4-pro backend, test wMAPE $\times 100$, three-split mean, seed 42). uni = per-channel, mv = joint forecast. CRAFTER shows a similar lift over RAW in both columns; the uni/mv difference comes from the backbone’s joint-forecast quality.

BB	RAW		CRAFTER	
	uni	mv	uni	mv
Toto	39.0	38.3	41.5	39.6
Moirai-2	27.1	39.2	26.2	38.8

a REFLECTION call only when fewer than three candidates pass the gate. Every system prompt is parameterized by a per-dataset domain preamble (the dataset name, a one-line domain hint, and a role-specific note). We show each role’s system and user templates below; run-time inputs appear as {placeholders} and the model must reply in strict JSON.

Planner (fast tier). *Inputs:* round index, validation-wMAPE trajectory, per-family coverage, under-served families, chronic failure subgroups, and rejection fingerprints by family. *Output:* exactly K exploration directions.

System. “You are a feature-exploration planner for {domain} time-series forecasting. Each round you see current feature coverage, chronic failure subgroups, recent rejection fingerprints grouped by family, and the wMAPE trend over recent rounds. Output K structured exploration directions; each tells the downstream Specifier

which family/subgroup to design for, what type of signal (combo, flag, or both), and which base feats to use. Return strict JSON {directions:[{family, target_subgroup, base_feat_pool, spec_type_hint, mechanism_hint, rationale}]}. Constraints: return exactly K directions; family from the allowed names; base_feat_pool a subset of the supplied feats; do not repeat a (family, subgroup) pair; prioritize under-served families and chronic subgroups.”

User. “Round {r}, $K=\{k\}$. wMAPE trajectory (last 5): {...}. Family names, coverage, under-served families, chronic subgroups (top 8), and rejections by family over the last 3 rounds: {...}. Existing dynamic features (top 30, with ρ): {...}. Available base feats: {...}. Output exactly K directions as JSON.”

Specifier (reasoning tier). *Inputs:* one Planner direction (family, optional subgroup, base-feat pool, signal-type hint, mechanism hint), up to eight chronic failure cases, and in-scope rejections to avoid. *Output:* 5–8 named specs (combo/flag/code).

System. “You are a feature engineer for {domain} time-series forecasting. You receive one exploration direction (family + optional target subgroup + allowed base-feat pool + mechanism hint). Strictly within that direction, produce 5–8 structured feature specs (combo, flag, or code). Hard constraints: combo/flag may use only feats in base_feat_pool; code specs may read the target history and the backbone forecast but may import only numpy, math, and scipy.{signal,stats,fft,special}, with no I/O, exec, eval, open, loops, or try; do not resubmit a spec identical to a rejected one on (op, feat set, transforms);

every spec must pass `validate_spec`. Output JSON under `missing_features`, each entry a combo (`feature_name`, `feat_a`, `feat_b`, `op`, `transform_a`, `transform_b`, `post_transform`), a flag (`conditions`, `combine`, `then`, `else`), or a code spec (`compute_func_code`). Guidance: the atomic search already covers plain combo/unary features, so prioritize flag (multi-condition thresholds the search cannot express) and code (FFT, autocorrelation, Hurst, windowed statistics); of the 5–8 specs keep $\text{combo} \leq 2$ and $\text{flag+code} \geq 60\%$.”

User. “family, target_subgroup, base_feat_pool, spec_type_hint, mechanism_hint: {...}. Chronic failure cases (for inspiration): {...}. Rejected fingerprints in this direction (avoid the same mechanism): {...}. Produce 5–8 specs as strict JSON under `missing_features`.”

Reflection (reasoning tier; fires only when <3 candidates pass the gate in a round). *Inputs:* one rejected candidate spec, its rejection reason (pair-rho-high or marginal-rho-low), and the direction’s mechanism hint. *Output:* exactly one rewritten spec with a different mechanism.

System. “You are a feature-improvement expert for {domain} time-series forecasting. You receive a candidate rejected by the novelty gate, its rejection reason, and the direction’s mechanism hint. Rewrite one new spec with a *different* mechanism—you must change at least one of `transform`, `feat_b`, or `op`; renaming alone does not count. Output JSON under `missing_features` containing exactly one spec.”

User. “Rejected candidate: {spec}. reject_reason: {...}. mechanism_hint: {...}. Rewrite one spec with a different mechanism.”

Base Proposer (invents new base columns that join the channel pool and become searchable next round). *Inputs:* round index, existing base-feature names, recent residual statistics, and chronic worst subgroups. *Output:* new univariate base-feature functions.

System. “You design new base-level features for {domain} time-series forecasting that feed the residual-correction GBDT. Each feature is a Python function `compute_NAME(history, chronos_pred)` returning one scalar, where `chronos_pred` is the backbone forecast. Allowed: `numpy` (as `np`), arithmetic, and simple control flow. Forbidden: any `import`, `I/O`, `exec/eval/open`, and referencing both `history` and `chronos_pred` in one function (each base feat must summarize a single input; cross-source interactions are left to the atomic search). Output strict JSON under `proposals`, each {feature_name, family, rationale, code} with family from the allowed set. Target patterns the existing base set under-represents and keep proposals mutually orthogonal.”

User. “Round {r}. Generate {n} new base features. Existing base names (avoid duplicates):

{...}. Recent residual stats: {...}. Chronic worst subgroups: {...}. Output strict JSON with a `proposals` list; each function takes (`history`, `chronos_pred`) and returns a float.”

I Framework Comparison: With vs. Without the LLM

Table S5 contrasts the CRAFT pipeline with and without the LLM agent at the component level. The two columns are the CRAFT-noLLM and CRAFT columns of Table S12; the counts are means per cell over the GPT-5.2 grid (selected = admitted to the corrector). Adding the compositional search on top of the covariate-only corrector moves accuracy by ≈ 0 (better-or-equal on 10/30 cells, mean $+0.006$ wMAPE); adding the LLM on top of the search is the decisive step (mean -0.037 wMAPE, better on 28/30 cells), and the gain concentrates on the weak-backbone cells where a semantic signal exists. At the level of accepted features, the LLM also dominates the final corrector: about three-quarters of admitted features are LLM-proposed, with LLM-combo features the most efficient and LLM-code features almost always pruned (per-source counts in Table S6, broken down by dataset and backbone in §J).

Table S5: Component-level comparison of CRAFT with vs. without the LLM agent. Counts are per-cell means over the GPT-5.2 grid.

Aspect	no-LLM (search only)	CRAFT (full)
Feature source	composed channels	+ LLM-flag/combo/code
Semantic naming	no	yes
Search guidance	syntactic UCB	LLM directions + UCB
LLM calls / round	0	~ 4
Interpretable names	composed exprs	named domain features
Selected features	~ 3.5 (all atomic)	~ 12.4 ($\sim 74\%$ LLM)
Gain vs. previous step	≈ 0 (10/30)	-0.037 wMAPE (28/30)
Where it helps	composed structure	weak backbone, semantic signal

J Feature-source anatomy by dataset and backbone

Table S6 aggregates the per-source *proposed* and *selected* feature counts and survival rates over the grid (the source for Fig. 3a); Tables S7 and S8 break them down by dataset and by backbone (GPT-5.2 fine-tuning grid, pooled over the two CRAFT

variants and three splits). Three patterns hold. The selected total rises with residual headroom: the weak-backbone, signal-rich FAVORITA cells admit the most features, while the small or saturated EPF and BIZITOBS cells sit at the floor. LLM-flag features are admitted mainly where the data has explicit calendar structure (FAVORITA); on EPF and BIZITOBS most are trimmed. LLM-code features are pruned almost everywhere. The mix is insensitive to the backbone (Moirai $\approx$ Moirai-2.0).

Table S6: **Feature-source anatomy (aggregate)**. Per-cell mean *proposed* (P) and *selected* (S, admitted to the corrector) feature counts by source, with survival rate S/P , over the fine-tuning grid (both Moirai backbones, three splits). LLM features supply about three-quarters of the selected set; LLM-code is almost always pruned.

Source	CRAFTER			CRAFTER [†]		
	P	S	surv	P	S	surv
atomic	5.7	3.5	61%	5.7	3.3	58%
LLM-flag	11.7	4.8	41%	10.8	4.7	44%
LLM-combo	5.7	3.9	68%	6.6	4.3	65%
LLM-code	2.3	0.2	9%	2.4	0.2	7%
Total	25.4	12.4	49%	25.5	12.5	49%
LLM share of S		72%			74%	

Table S7: Feature-source anatomy by dataset (per-cell mean, proposed/selected). P , S are the totals; $LLM\%(S)$ is the LLM share of selected features.

Dataset	atomic	flag	combo	code	P	S	$LLM\%(S)$
EPF	5.5/3.1	9.2/2.8	6.8/4.1	2.6/0.0	24.1	10.0	69%
ROSSMANN	5.7/3.3	11.2/4.2	6.6/3.8	1.6/0.1	25.1	11.5	71%
BIZITOBS	5.3/2.5	9.2/2.0	7.8/4.9	2.3/0.6	24.6	10.0	75%
FAVORITA	6.0/4.0	12.1/7.2	6.4/4.8	2.9/0.0	27.4	16.0	75%

Table S8: Feature-source anatomy by backbone (per-cell mean, proposed/selected).

Backbone	atomic	flag	combo	code	P	S	$LLM\%(S)$
Moirai	5.8/3.4	11.3/4.4	6.2/4.7	2.3/0.2	25.6	12.6	73%
Moirai-2.0	5.6/3.4	11.2/5.1	6.1/3.5	2.4/0.2	25.3	12.2	72%

K Representative Accepted Features

The examples below illustrate what each source contributes. They are drawn from the per-feature audit logs of the DeepSeek CRAFTER [†] runs (rolling split 8, seed 42), which record every candidate’s acceptance correlation ρ and its split gain in the final corrector. “Share” is the feature’s fraction of its cell’s total corrector gain; gains are target-scale dependent, so shares compare only within a cell.

Atomic. Compositions the search assembled from raw channels; the recorded name encodes the expression, e.g., `binary_safe_div_log-cov_Promo_softplus-dow`. That feature, `safe_div(log(Promo), softplus(dow))` (ROSSMANN, Moirai-2; $\rho=0.61$, share 50%), is a promotion-by-weekday interaction, and the identical spec is also accepted in the decoupled CRAFTER run of the same cell. `min(softplus(hour), is_weekend)` (BIZITOBS, Timer; $\rho=-0.78$, share 48%) gates the hourly IT-traffic residual by weekend and time of day. `min(log(holiday), tanh(is_weekend))` (ROHLIK) is selected under four of the five backbones; for a tree corrector it reduces to a re-derived holiday indicator, typical of how the search re-expresses information the schema already carries rather than adding semantics.

LLM-combo. Named combinations with a stated mechanism. `renewable_penetration`, the renewable-generation forecast over the load forecast (EPF, Moirai; $\rho=-0.81$, share 81%), was proposed because “values above 1.0 directly signal surplus conditions ... which modulates price volatility”; a renewables-over-load variant is selected on all five EPF backbones. `baseline_mean_x_open`, the context mean $\times$ Open (ROSSMANN, Moirai; share 88%), is the expected sales level when the store is open. `event_ratio`, the last value over the context mean (FAVORITA, Chronos; share 37%), was proposed to flag “sudden drops typical of earthquake shocks”.

LLM-flag. The LLM proposes the regime and its channels; the numeric thresholds are pool quantiles filled in by the enumerator. On ROSSMANN (Moirai-2; share 14%) an anomaly direction targeting abnormal promotion days yields $1[\text{ctx_std}>q_{75} \wedge \text{Promo}>0 \wedge \text{Open}\geq 1]$, an open, high-volatility store running a promotion. On BIZITOBS (Toto; $\rho=-0.61$, share 18%), $1[\text{is_weekend} \vee h<6]$ marks the low-load regime of the API-traffic series. On FAVORITA (Moirai), $1[\text{onpromotion}\leq 0 \wedge \text{holiday}\leq 0]$ separates quiet baseline days from event days; holiday-conditioned flags are selected on four of the five FAVORITA backbones.

LLM-code. Selected code features are rare (Table S6), and the survivors compute statistics the grammar cannot express. `trend_r_squared` (FAVORITA, Toto; share 55%) fits a line to the last 28 days and returns its R^2 , proposed as “how

strongly a linear model explains recent sales, complementary to slope magnitude”. `mk_trend_tau` (M5, Timer; share 28%) is Kendall’s τ of demand against time, a monotone-trend strength robust to outliers; its accepted source is

```
def compute_mk_trend_tau(history, chronos_pred):
    import numpy as np
    from scipy import stats
    if len(history) < 5:
        return 0.0
    tau, p = stats.kendalltau(
        np.arange(len(history)), history)
    return float(tau) if not np.isnan(tau) else 0.0
```

L Feature transferability (faithful replay)

We test whether features learned on one backtest window stay useful on a later one. For an ordered split pair $S \rightarrow T$ we load the accepted feature specifications (combinations and synthesized code) from the source cell, force-carry them into the target split’s accepted set, re-materialize the code features on the target data, and refit and evaluate the corrector with no new proposals (LLM and search off, validation gating on). Two references on the same target bound the effect: *self*, the target’s own accepted features replayed back (a faithfulness upper bound that should reproduce native CRAFT), and *cov*, a covariate-only corrector (the no-transfer lower bound). The grid is two datasets (EPF, BIZITOBS) $\times$ five backbones $\times$ three split pairs ($6 \rightarrow 7$, $7 \rightarrow 8$, $6 \rightarrow 8$) $\times$ {cov, self, xfer} = 90 cells, all completing. The self column at split S reproduces native CRAFT closely, so the replay is faithful. Table S9 summarizes the 30 source-cell triples and Table S10 lists every cell.

Three findings hold. Transferred features match the target’s own at the median (rel. +0.2%, 17/30 within $\pm 5\%$); the mean is pulled up only by a heavy tail. Distance decay is mild: the median for the two-split jump ($6 \rightarrow 8$) is only a couple of percent above the adjacent-split median. The 7/30 catastrophic transfers (worse than self by $> 15\%$) concentrate on the single-series EPF and on Moirai backbones—the shift-sensitive corner of §5.2—while the multivariate BIZITOBS panel transfers with median +0.0%. Low feature-name overlap across splits therefore reflects equivalent-feature redundancy, not non-transferability.

M Feature-budget sweep

Table S11 sweeps the feature cap K over the 30 GPT-5.2 cells (each a three-split mean), backing Fig. 2b and the budget claim of §5.1. The externals here are the honest budget-50 sweep

Table S9: Transferability summary over the 30 source-cell triples. “rel.” is the relative test wMAPE gap to the reference (negative favors transfer).

Question	Metric	Result
xfer vs. self (upper bound)	median rel.	+0.2%; 17/30 within $\pm 5\%$
xfer vs. cov (lower bound)	median rel.	+0.0%; 12/30 beat cov
distance decay	adj. vs. far	+0.0% vs. +1.7% (mild)
catastrophic ($> 15\%$)	count	7/30, EPF/Moirai-dominated
by dataset	median rel. to self	EPF +5.6%, BIZITOBS +0.0%

(`grid_allbb_uncap`), so their $K \geq 10$ columns are genuinely re-generated rather than repeats of the small cap. The win rate over RAW is constant at 80% across every K : the budget never changes which cells CRAFT wins, only by how much. The margin over the best external system instead widens with K and then saturates at $K=20$ ($K=20/50$ /all are nearly identical), so $K=20$ is the practical operating point. Split by residual headroom, the budget amplifies an existing gap rather than creating one: on raw-weak cells (external moves RAW by $\geq 2\%$) the mean lift over RAW reaches -0.131 at $K=20$ (20/20 cells), whereas on raw-strong cells it sits near $+0.008$ and a larger cap, if anything, fits more noise.

N Full per-cell results

Table S12 gives every cell of the GPT-5.2 main grid (the source for Fig. 2a); Table S13 is the DeepSeek-3.4-pro reproduction. Figure S1 plots the per-cell gain against backbone competence: the lift over RAW trends positive with the backbone’s own error but is noisy, which is why the body reports the cleaner per-backbone aggregate (Fig. 2a) rather than the raw scatter. A single quantity—how much exploitable residual the frozen forecast leaves—orders the gain: the wrapper is net-positive on the weak, high-error cells to the right and neutral-to-negative on the saturated, low-error cells to the left, with no separate dependence on dataset or on which foundation model produced the point. The per-cell scatter is the headroom condition of §5.2 before averaging.

Table S10: Per-cell faithful replay (test wMAPE $\times 100$, lower is better). *cov*: covariate-only target corrector (no-transfer lower bound); *self*: target’s own accepted features replayed (faithfulness upper bound); *xfer*: source-split features replayed on the target. Δ_s , Δ_c are the relative gaps of xfer to self and to cov in %. Pairs 7 $\rightarrow$ 8 and 6 $\rightarrow$ 8 share the same target (split 8), so their cov/self match.

BB	pair	cov	self	xfer	Δ_s	Δ_c
EPF						
tim	6 $\rightarrow$ 7	62.8	63.9	62.8	-1.7	+0.0
tim	7 $\rightarrow$ 8	23.9	23.2	23.4	+0.9	-2.4
tim	6 $\rightarrow$ 8	23.9	23.2	23.8	+2.9	-0.5
chr	6 $\rightarrow$ 7	62.3	62.6	66.1	+5.6	+6.0
chr	7 $\rightarrow$ 8	26.3	26.3	25.9	-1.4	-1.6
chr	6 $\rightarrow$ 8	26.3	26.3	26.4	+0.5	+0.3
moi	6 $\rightarrow$ 7	59.0	65.9	115.3	+75.0	+95.5
moi	7 $\rightarrow$ 8	25.0	19.8	22.2	+11.8	-11.5
moi	6 $\rightarrow$ 8	25.0	19.8	47.5	+139.5	+89.6
toto	6 $\rightarrow$ 7	61.5	61.2	63.1	+3.0	+2.5
toto	7 $\rightarrow$ 8	23.7	19.7	21.9	+11.1	-7.4
toto	6 $\rightarrow$ 8	23.7	19.7	22.8	+15.7	-3.6
moi ₂	6 $\rightarrow$ 7	62.0	90.2	61.1	-32.3	-1.6
moi ₂	7 $\rightarrow$ 8	24.3	19.5	55.1	+182.0	+126.8
moi ₂	6 $\rightarrow$ 8	24.3	19.5	23.3	+19.2	-4.1
BIZITOBS						
tim	6 $\rightarrow$ 7	31.8	33.3	31.1	-6.7	-2.3
tim	7 $\rightarrow$ 8	30.5	31.7	30.8	-3.0	+0.9
tim	6 $\rightarrow$ 8	30.5	31.7	30.2	-4.8	-0.9
chr	6 $\rightarrow$ 7	22.6	22.6	22.6	+0.0	+0.0
chr	7 $\rightarrow$ 8	21.7	21.7	21.7	+0.0	+0.0
chr	6 $\rightarrow$ 8	21.7	21.7	21.7	+0.0	+0.0
moi	6 $\rightarrow$ 7	36.2	38.0	36.5	-4.0	+0.8
moi	7 $\rightarrow$ 8	32.1	32.2	40.4	+25.2	+25.8
moi	6 $\rightarrow$ 8	32.1	32.2	35.7	+10.9	+11.4
toto	6 $\rightarrow$ 7	33.8	31.7	31.6	-0.2	-6.4
toto	7 $\rightarrow$ 8	30.3	31.3	30.1	-3.9	-0.5
toto	6 $\rightarrow$ 8	30.3	31.3	30.6	-2.3	+1.2
moi ₂	6 $\rightarrow$ 7	24.8	24.8	29.0	+16.6	+16.6
moi ₂	7 $\rightarrow$ 8	23.9	23.9	23.9	+0.0	+0.0
moi ₂	6 $\rightarrow$ 8	23.9	23.9	23.9	+0.0	+0.0

Table S11: Feature-budget sweep (GPT-5.2, 30 cells, three-split mean). Win rates and mean differences in test wMAPE for CRAFT (decoupled) vs. RAW and vs. the best external system (negative mean favors CRAFT).

K	CRAFT < RAW	mean(-RAW)	CRAFT < best-ext	mean(-ext)
5	80%	-0.067	60%	-0.020
10	80%	-0.073	73%	-0.025
20	80%	-0.085	73%	-0.038
50	80%	-0.085	73%	-0.039
all	80%	-0.085	73%	-0.036

Algorithm S1 One CRAFT round

Require: accepted features $\mathcal{F}$, arm statistics Θ , channel pool $\mathcal{C}$, frozen backbone forecast $\hat{y}$

- 1: fit the corrector set on $\mathcal{F}$; keep the head (NONE/additive/multiplicative) with lowest validation wMAPE {NONE floors risk, §3.4}
- 2: $r \leftarrow$ unexplained validation residual under the kept corrector; compute family coverage
- 3: $D \leftarrow \text{PLANNER}(\text{coverage}, \mathcal{F}, \text{blacklist})$ {Reasoning: exploration directions, §3.2}
- 4: $P_R \leftarrow (\bigcup_{d \in D} \text{SPECIFIER}(d)) \cup \text{BASEPROPOSER}()$ {Reasoning: LLM-named features}
- 5: $P_E \leftarrow \text{UCB-SAMPLE}(\Theta, \mathcal{C})$ {Exploration: compositional search, §3.2}
- 6: $\mathcal{A} \leftarrow \{c \in \text{dedup}(P_E \cup P_R) : |\rho(c, r)| > \tau, \text{ non-collinear, within budgets}\}$ {source-blind gate, §3.3}
- 7: $\mathcal{C} \leftarrow \mathcal{C} \cup \{\text{new base columns in } \mathcal{A}\}$ {Reasoning $\rightarrow$ Exploration coupling (off by default)}
- 8: refit the corrector on $\mathcal{F} \cup \mathcal{A}$; $\mathcal{T} \leftarrow$ features below the per-tier top- K by permutation Δ val-wMAPE {prune}
- 9: update Θ with UCB credit from kept vs. trimmed importances and the validation-wMAPE gain
- 10: **return** $(\mathcal{F} \cup \mathcal{A}) \setminus \mathcal{T}, \Theta, \mathcal{C}$

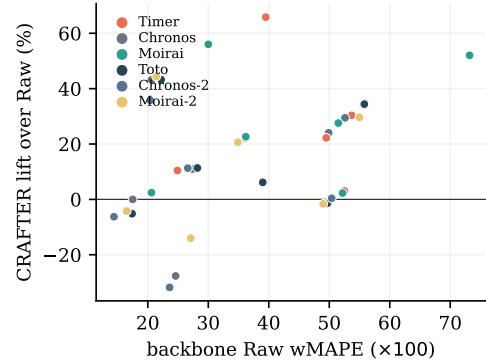


Figure S1: **Gain vs. backbone competence (per cell, GPT-5.2).** Each point is one (dataset, backbone) cell: x is the frozen backbone’s Raw wMAPE $\times 100$, y is CRAFT’s lift over Raw. The gain trends positive with backbone error (more exploitable residual) but is noisy across datasets; the per-backbone means (Fig. 2a) summarize the trend. Residual headroom, not dataset or backbone identity, orders the gain: points on the high-error right are consistently corrected, while saturated low-error points on the left cluster near zero or below, the per-cell form of the headroom condition (§5.2).

Table S12: **Main results.** CRAFT (two rightmost, gray columns) vs. the frozen backbone (RAW), a head-only backbone fine-tune (FT), a covariate-only corrector (COV), the no-LLM corrector (noLLM), and three external feature-engineering systems, on the $6 \times 6 = 36$ -cell grid (test wMAPE $\times 100$, lower is better). Every feature-engineering method uses the **same GPT-5.2 backend** and harness, so only the feature *generator* differs. The noLLM, external, and CRAFT columns average three rolling splits $\times$ three seeds; RAW, COV, and FT involve no LLM or search randomness and are single-run on the same splits. Per-cell seed variability is quantified on the DeepSeek companion grid (Table S13 caption; App. D). CHR is Chronos (univariate); CHR₂ is the covariate-aware Chronos-2, reported single-seed (42) on the same splits. CRAFT is the full system; CRAFT[†] adds source coupling (§5.2 of the main paper). **Bold** = a CRAFT variant beats *every* non-fine-tuned baseline in that row (both may bold); if neither does, the row’s best method is bold. A star (*) marks a bold CRAFT variant that *also* beats FT. RAW, COV and FT are LLM-independent and shared with Table S13; the stronger full-parameter Moirai fine-tune is in Table S2.

Dataset	BB	RAW	FT	COV	noLLM	TSFRESH	CAAFE	LLM-FE	CRAFT	CRAFT [†]
epf	tim	53.7	56.9	46.2	48.9	44.7	49.9	50.5	37.4*	37.1*
	chr	49.9	52.0	45.4	42.2	46.5	46.5	47.5	37.9*	31.6*
	moi	73.2	71.2	60.7	62.7	60.3	62.0	58.7	35.1*	35.3*
	toto	55.8	55.8	49.8	49.9	49.9	45.1	46.2	36.6*	32.7*
	chr ₂	52.6	55.1	43.7	41.3	50.6	42.4	44.0	37.1*	32.2*
	moi ₂	55.0	55.0	41.6	42.1	42.3	43.9	42.0	38.7*	40.0*
rossmann	tim	39.5	39.5	16.7	16.7	13.9	17.2	17.2	13.5*	13.7*
	chr	20.6	21.3	11.9	11.9	12.2	12.0	12.0	11.7*	11.7*
	moi	30.0	29.8	24.7	26.4	24.8	24.8	24.8	13.2*	13.1*
	toto	22.2	22.2	14.5	14.5	13.9	14.4	14.3	12.6*	12.5*
	chr ₂	20.4	21.1	15.9	15.8	15.5	15.9	15.9	13.1*	12.8*
	moi ₂	21.4	21.4	12.6	12.7	12.8	12.6	12.6	11.9*	12.0*
rohlik	tim	36.1	30.3	33.4	33.4	33.3	32.9	32.8	28.1*	28.1*
	chr	27.4	27.2	24.7	24.6	24.6	24.7	24.6	24.4*	24.4*
	moi	36.2	31.2	33.5	33.5	33.4	32.8	32.7	28.0*	28.0*
	toto	28.2	28.2	25.1	25.1	25.1	25.2	25.2	25.0*	24.9*
	chr ₂	26.6	27.1	23.8	23.8	23.8	23.9	23.9	23.6*	23.6*
	moi ₂	34.9	29.2	32.2	32.3	32.1	31.9	31.8	27.7*	27.7*
bizitobs	tim	49.5	51.9	42.4	43.2	42.5	43.6	43.6	38.5*	37.4*
	chr	24.6	25.0	25.7	34.1	25.7	25.7	25.7	31.4	30.3
	moi	51.5	50.9	43.5	43.9	43.0	45.1	44.2	37.3*	37.5*
	toto	39.0	39.0	40.9	37.6	38.9	40.8	40.8	36.6*	35.6*
	chr ₂	23.6	25.3	31.0	30.9	24.4	24.4	24.4	31.1	31.4
	moi ₂	27.1	27.1	27.7	29.6	27.9	28.0	28.0	30.9	29.6
favorita	tim	24.9	23.2	21.8	23.1	21.5	21.7	21.6	22.3	22.7
	chr	17.5	17.4	17.6	18.0	17.4	17.6	17.6	17.5	17.4
	moi	20.6	20.6	19.8	20.3	19.9	19.8	19.6	20.1	20.3
	toto	17.4	17.4	17.7	18.2	17.4	17.6	17.6	18.3	18.2
	chr ₂	14.4	14.6	15.2	15.3	14.5	14.7	14.6	15.3	15.9
	moi ₂	16.5	16.5	16.6	17.4	16.5	16.6	16.6	17.2	17.2
m5	tim	52.5	51.4	51.0	51.0	51.0	51.0	51.0	50.9*	50.9*
	chr	50.3	49.9	50.3	50.7	50.3	50.3	50.3	50.1	50.2
	moi	52.2	52.2	52.1	52.3	52.1	52.1	52.1	51.0*	51.6*
	toto	49.7	49.7	50.1	50.5	50.1	50.1	50.0	50.4	50.4
	chr ₂	50.4	49.5	51.2	51.2	50.6	50.6	50.5	50.2	50.3
	moi ₂	49.0	49.0	49.3	49.9	49.3	49.3	49.3	49.8	49.8

Table S13: **DeepSeek-3.4-pro full grid** (cross-LLM reproduction; test wMAPE $\times 100$, $6\times 6=36$ cells, lower is better). Companion to the GPT-5.2 Table S12 with the same columns; the per-backbone summary is in Table 1 of the main paper. RAW, COV and FT are LLM-independent and shared with Table S12. noLLM, CRAFTER and the externals are the DeepSeek runs; CRAFTER[†] is the DeepSeek source-coupling variant (3 splits, single-seed). **Bold** and star (*) follow Table S12. Across the three seeds the median per-cell std of the split-averaged value is 0.02 (noLLM), 0.01 (TSFRESH), 0.05 (CAAFE), 0.08 (LLM-FE), and 0.49 (CRAFTER); RAW is seed-deterministic, and the largest CRAFTER stds all sit on EPF and BIZITOBS. The CHR₂ (Chronos-2) rows are single-seed (42) on the same three splits, matching how Chronos-2 is reported in Table S12.

Dataset	BB	RAW	FT	COV	noLLM	TSFRESH	CAAFE	LLM-FE	CRAFTER	CRAFTER [†]
epf	tim	53.7	56.9	46.2	43.7	44.7	50.1	53.4	43.4*	31.3*
	chr	49.9	52.0	45.4	42.6	45.2	47.6	45.7	48.5	43.9
	moi	73.2	71.2	60.7	57.2	56.6	60.7	57.5	59.2	28.6*
	toto	55.8	55.8	49.8	50.3	49.9	45.6	45.9	43.9*	40.9*
	chr ₂	52.6	55.1	43.7	47.0	42.9	43.6	44.3	46.8	37.2*
	moi ₂	55.0	55.0	41.6	42.4	42.3	45.1	42.8	44.4	36.0*
rossmann	tim	39.5	39.5	16.7	17.0	13.8	17.4	17.4	14.2	13.2*
	chr	20.6	21.3	11.9	11.5	12.3	11.9	11.9	11.5*	11.5*
	moi	30.0	29.8	24.7	25.0	24.8	24.8	24.8	23.9*	13.2*
	toto	22.2	22.2	14.5	13.9	13.9	14.3	14.4	13.9*	12.0*
	chr ₂	20.4	21.1	15.9	16.4	16.0	15.9	16.0	16.3	12.6*
	moi ₂	21.4	21.4	12.6	12.4	12.8	12.6	12.6	11.9*	12.1*
rohlik	tim	36.1	30.3	33.4	26.8	26.4	26.1	26.1	26.7	26.6
	chr	27.4	27.2	24.7	24.6	24.6	24.7	24.6	24.5*	24.3*
	moi	36.2	31.2	33.5	33.6	33.4	32.9	32.6	32.5	28.4*
	toto	28.2	28.2	25.1	25.1	25.1	25.2	25.2	25.0*	24.9*
	chr ₂	26.6	27.1	23.8	23.5	23.5	23.5	23.5	23.4*	23.5*
	moi ₂	34.9	29.2	32.2	32.4	32.1	31.9	31.8	31.5	28.0*
bizitobs	tim	49.5	51.9	42.4	43.9	42.5	43.7	44.6	42.5	35.7*
	chr	24.6	25.0	25.7	28.2	25.7	25.7	25.7	24.6*	26.7
	moi	51.5	50.9	43.5	44.8	43.1	42.6	42.7	43.3	38.4*
	toto	39.0	39.0	40.9	44.0	39.0	41.6	41.0	40.4	36.6*
	chr ₂	23.6	25.3	31.0	23.6	24.4	24.4	24.4	25.9	25.8
	moi ₂	27.1	27.1	27.7	27.6	27.9	28.4	28.2	26.7*	26.6*
favorita	tim	24.9	23.2	21.8	21.3	21.7	21.6	21.5	21.1*	21.0*
	chr	17.5	17.4	17.6	17.6	17.4	17.6	17.6	17.5	17.6
	moi	20.6	20.6	19.8	19.8	20.0	19.8	19.8	19.9	19.9
	toto	17.4	17.4	17.7	17.6	17.4	17.6	17.5	17.5	17.4
	chr ₂	14.4	14.6	15.2	14.6	14.5	14.7	14.6	14.4*	14.5
	moi ₂	16.5	16.5	16.6	16.6	16.5	16.6	16.6	16.6	16.3*
m5	tim	52.5	51.4	51.0	51.1	51.0	50.9	50.9	51.1	52.4
	chr	50.3	49.9	50.3	50.3	50.3	50.3	50.3	50.3	51.1
	moi	52.2	52.2	52.1	52.1	52.1	52.0	52.0	52.2	52.0*
	toto	49.7	49.7	50.1	49.7	50.1	50.1	50.1	49.7	49.9
	chr ₂	50.4	49.5	51.2	49.6	49.7	49.6	49.6	49.6	49.3*
	moi ₂	49.0	49.0	49.3	49.1	49.3	49.3	49.3	49.1	49.4